CO4 Assignment 4: Structures, Unions & Storage Classes

Course: Problem Solving & Programming in C **Student Name:** [Student Name] | **Roll No:** [Roll Number]

Task 1 — Data Design Review: Primary Health Centre Register

Task 1.1 — Diagnosis of Design Deficiencies

1. **Fragmented Parallel Arrays:** Storing `name`, `age`, `phone`, `bp`, and `visitDate` in five independent arrays decouples fields belonging to the same patient. Sorting by age requires manually swapping five arrays in lockstep; missing one array corrupts patient records across the clinic.
2. **Buffer Overflow in `char phone[100][10]`:** A 10-digit phone number requires 11 bytes (10 digits + null terminator `'\0'`). `strcpy(phone[count], p)` writes `'\0'` out of bounds into the next patient's phone number, corrupting contact records.
3. **Lossy Representation of Blood Pressure in `float bp[100]`:** Blood pressure consists of two distinct integer measurements (systolic and diastolic, e.g., 120/80 mmHg). Storing it as a single `float` either loses the diastolic value or forces a division (`1.5f`), making it impossible to distinguish 120/80 from 150/100.
4. **Undersized Text Date `char visitDate[100][8]`:** Standard date strings like `"12/10/2026"` require 11 bytes (10 chars + `'\0'`). Even compact `"12102026"` needs 9 bytes, causing `strcpy` buffer overflow and preventing chronological comparisons.
5. **Missing Capacity Bounds Check & Global State:** `addPatient()` increments global `count` without checking `count < 100`, causing silent out-of-bounds writes after 100 patients and preventing multiple independent clinic registers.

Task 1.2 — Repaired Structure-Based Design

```
#include <stdio.h>
#include <string.h>

#define MAX_PATIENTS 100

typedef struct { int day, month, year; } Date;
typedef struct { int systolic, diastolic; } BloodPressure;

typedef struct {
    char name[40];
    int age;
    char phone[12];       /* 10 digits + '\0' + spare byte */
    BloodPressure bp;
    Date visitDate;
} Patient;

int addPatient(Patient reg[], int *count, const Patient *new_p) {
    if (*count >= MAX_PATIENTS) return 0; /* Overflow prevented */
    reg[*count] = *new_p;                 /* Single atomic struct copy */
    (*count)++;
    return 1;
}
```

Task 1.3 — Design Defence & Alternative Complexity Analysis

Design Defence (74 words): Encapsulating all fields inside a `Patient` structure guarantees data cohesion: swapping two patients during an age sort requires moving a single structure rather than five synchronized arrays. Passing a `const Patient *` pointer avoids copying entire records onto the stack and enforces bounds checking via `*count` instead of vulnerable global state. Finally, adding new clinical attributes (such as blood group or diagnosis) requires updating only the `Patient` definition without altering function signatures.

Parallel Arrays vs. Array of Structures (AoS): Parallel arrays (Structure of Arrays, SoA) are preferred in SIMD/vectorized analytics or cache-sensitive scans over a single field (e.g., computing average age across 100,000 patients without loading 40-byte names into CPU cache lines). When sorting n patients ($O(n\ log\ n)$ or $O(n^2)$ swaps), AoS performs 1 contiguous block swap per exchange, whereas 5 parallel arrays require 5 separate element swaps. On a 64-bit system, `Patient` occupies $40 + 4 + 12 + 8 + 12 = 76$ bytes (zero internal padding because all members are 4-byte aligned), matching the 76 bytes per record of parallel arrays.

Task 2 — From Requirement to Nested Structures: Courier Tracking

Task 2.1, 2.2 & 2.4 — Entity Model, Definitions & Access Paths

- **Entities & Nesting:** `Date` (day, month, year) and `Address` (name, city, pincode) are standalone value structures nested inside `Parcel` (id, weight, service_type, charge, sender, receiver, booked).
- **Access Paths (Third Parcel, Index 2):**
  (i) Array + dot operator: `list[2].receiver.pincode`
  (ii) Pointer + arrow operator (given `struct parcel *ptr = &list[2];`): `ptr->receiver.pincode` (or `(ptr + 2)->receiver.pincode` if `ptr = list`).

```
#include <stdio.h>

typedef struct { int day, month, year; } Date;
typedef struct { char name[30]; char city[20]; int pincode; } Address;
typedef enum   { NORMAL, EXPRESS } ServiceType;

typedef struct {
    int id;
    float weight;
    Address sender;
    Address receiver;
    Date booked;
    ServiceType service;
    float charge;
} Parcel;

Parcel daily_bookings[200];
```

Task 2.3 & Completed Code Execution

```
/* Pseudocode:
   (a) SortByWeight(P, n): For i = 0..n-2, find min_idx in i..n-1 with smallest P[j].weight;
       swap P[i] and P[min_idx].
   (b) ListByCity(P, n, target_city): For i = 0..n-1, if strcmp(P[i].receiver.city,
target_city) == 0,
       print P[i]. */

#include <stdio.h>

struct date    { int day, month, year; };
struct address { char city[20]; int pincode; };
struct parcel  { int id; float weight; struct address to; struct date booked; };

void print_parcel(const struct parcel *p) {
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min_idx].weight) min_idx = j;
        if (min_idx != i) {
            struct parcel tmp = p[i];
            p[i] = p[min_idx];
            p[min_idx] = tmp;
        }
    }
}

int main(void) {
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001},    {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077},    {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };
    sort_by_weight(list, 3);
    for (int i = 0; i < 3; i++) print_parcel(&list[i]);
    printf("Bytes copied per swap: %zu\n", sizeof(struct parcel));
    return 0;
}
```

Memory & Flat Structure Analysis: `sizeof(struct parcel)` is **44 bytes** (`4 + 4 + [20 + 4] + [4 + 4 + 4] = 44` bytes), so each swap copies 44 bytes (132 bytes across 3 assignments). Flattening `address` and `date` directly into `parcel` produces the identical 44-byte footprint, but degrades modularity—duplicating `sender_city`, `sender_pincode`, `receiver_city`, `receiver_pincode` and preventing reusable functions like `print_address(const struct address *)`.

Task 3 — Unions and Memory Layout: Weather-Station Packets

Task 3.1, 3.2 & Completed Code

- **Design Justification & Size Prediction:** A plain `struct` allocates memory for all three sensor fields simultaneously ($4 + 4 + 4 + 8 = 20$ bytes), wasting 60% of RAM per packet because each packet carries only one reading. A **tagged union** overlays `celsius` (4 B), `percent` (4 B), and `status[8]` (8 B) at the exact same memory offset. Thus, `sizeof(union)` equals its largest member (**8 bytes**), and `sizeof(struct reading)` equals `sizeof(enum kind)` (4 bytes) + `sizeof(union)` (8 bytes) = **12 bytes** (zero padding needed since 8 is a multiple of 4-byte alignment).

```
#include <stdio.h>

enum kind { TEMPERATURE, HUMIDITY, STATUS };

struct reading {
    enum kind type;
    union {
        float celsius;
        int percent;
        char status[8];
    } value;
};

void show(const struct reading *r) {
    switch (r->type) {
        case TEMPERATURE: printf("Temp %.1f C\n", r->value.celsius); break;
        case HUMIDITY:    printf("Humidity %d%%\n", r->value.percent); break;
        case STATUS:      printf("Status %s\n", r->value.status); break;
    }
}

int main(void) {
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY,    {.percent = 68}},
        {STATUS,      {.status = "OK"}}
    };
    printf("sizeof union = %zu, sizeof struct reading = %zu\n",
           sizeof log[0].value, sizeof log[0]);
    for (int i = 0; i < 3; i++) show(&log[i]);

    /* Task 3.4 Misuse Experiment */
    struct reading mis = {TEMPERATURE, {.celsius = 31.5f}};
    printf("Misuse (31.5f read as int percent): %d\n", mis.value.percent);
    return 0;
}
```

- **Task 3.4 Misuse Experiment Result:** Storing `31.5f` writes the IEEE-754 single-precision bit pattern `0x41FC0000` into the union. Reading `mis.value.percent` reinterprets those exact 32 bits as a signed two's complement integer, outputting **`1107034112`** —garbage data proving why the `type` tag must always be checked before reading a union.
- **Alternative Complexity Analysis:** Three separate arrays eliminate the 4-byte tag per element (4–8 B/packet) and allow `O(1)` access, but lose chronological arrival ordering across mixed sensor types. Allocating each packet with `malloc` and storing `void *` pointers uses `O(1)` pointer lookup, but adds 8 bytes per pointer plus ~16–32 bytes of heap allocator metadata per packet (32–48 bytes total) and destroys cache locality.

Task 4 — Structure Forensics: Savings Account Interest

Task 4.1–4.4 — Defect Localization, Address Proof & Repaired Code

- **Defect 1 (Balances Unchanged):** `addInterest(struct account a, float rate)` receives `a` **by value**. C copies `list[i]` into a separate stack frame parameter `a`; modifying `a.balance` alters only the temporary copy, which is discarded upon return.
- **Defect 2 (Compile Error):** In `printf("%d %f\n", p.number, p.balance);`, `p` is a pointer (`struct account *`), not a structure. Applying the dot operator `.` to a pointer causes a compiler error (`request for member 'number' in something not a structure or union`); it must use `p->number` and `p->balance`.

```
#include <stdio.h>

struct account { int number; float balance; };

void addInterest(struct account *a, float rate) {
    printf("  [Inside addInterest] address of *a = %p\n", (void *)a);
    a->balance = a->balance + a->balance * rate / 100.0f;
}

int main(void) {
    struct account list[3] = {{101, 5000.0f}, {102, 12000.0f}, {103, 800.0f}};
    struct account *p = list;
    for (int i = 0; i < 3; i++) {
        printf("[In main] address of list[%d] = %p\n", i, (void *)&list[i]);
        addInterest(&list[i], 4.0f);
    }
    printf("First account after interest: %d %.2f\n", p->number, p->balance);
    return 0;
}
```

- **Address Evidence & Passing Guidelines:** In the buggy version, `&a` inside `addInterest` (e.g., `0x7ffee4b0`) differs from `&list[0]` (`0x7ffee4d0`); in the repaired version, both print the exact same address. **Guideline:** Pass *by value* only for tiny immutable structs (≤ 16 bytes, like `Date`); pass *by pointer* (`T *`) when the function must mutate the caller's object; pass *by* `const` *pointer* (`const T *`) for read-only access to larger structs to avoid copying overhead while guaranteeing immutability.

- **Alternative Return-by-Value Analysis:** Writing `list[i] = addInterest(list[i], 4)` copies 8 bytes (`sizeof(struct account)`) into the argument and 8 bytes back on return (16 bytes copied per call + stack frame allocation), whereas passing a pointer copies a single 8-byte address and writes in place. For large structs (e.g., 256 bytes), return-by-value copies 512 bytes per call vs. 8 bytes for a pointer.

Task 5 — Storage Classes: Hostel Mess Token Counter

Task 5.1–5.3 — Four Implementations, Two-Counter Refactor & Comparison

```
#include <stdio.h>

/* (a) Automatic local: resets to 1000 every call -> outputs 1001, 1001, 1001 */
int auto_token(void) { int token = 1000; return ++token; }

/* (b) Static local: retains state across calls -> outputs 1001, 1002, 1003 */
int next_token(void) { static int token = 1000; return ++token; }

/* (c) Global across two files: file1.c defines `int g_token = 1000;`
       file2.c declares `extern int g_token; int ext_token(void) { return ++g_token; }` */

int main(void) {
    int s1 = next_token(), s2 = next_token(), s3 = next_token();
    int a1 = auto_token(), a2 = auto_token(), a3 = auto_token();
    printf("static: %d %d %d\n", s1, s2, s3);
    printf("auto: %d %d %d\n", a1, a2, a3);

    /* (d) register loop variable */
    for (register int i = 0; i < 3; i++) printf("Loop %d: %d\n", i, next_token());
    return 0;
}
```

Storage Class	Scope	Lifetime	Default Init	Risk & Two-Counter Effort (Veg / Non-Veg)
`auto` (local)	Block	Function call	Garbage	Cannot persist count; fails requirement.
`static` (local)	Block	Entire program	`0`	Low risk; requires duplicating function (`next_veg` , `next_nonveg`) or passing a flag.
`extern` (global)	All files	Entire program	`0`	High accidental overwrite risk; requires two globals (`veg_tok` , `nonveg_tok`).
`register`	Block	Block	Garbage	Hint to store in CPU register; `&i` is illegal because registers have no RAM address.

Unspecified Evaluation Order in `printf` & Pointer Alternative: In `printf("%d %d %d", next_token(), next_token(), next_token())` , C does not specify whether function arguments are evaluated left-to-right or right-to-left; GCC commonly evaluates right-to-left, printing `1003 1002 1001` instead of `1001 1002 1003` . Passing `int next_token(int *counter)` keeps `O(1)` time, ties counter lifetime to the caller's scope, and supports unlimited independent counters (`&veg_cnt` , `&nonveg_cnt`) with zero code duplication.

Task 6 — Design Challenge: College Canteen Inventory & Billing

Stage 1–4 — Data Model, Critical Decisions & Pseudocode

- **Entities & Data Model:** `Date` (day, month, year), `Time` (hour, min), `ItemType` (`PACKAGED` , `COOKED` , `COMBO`), and a tagged union inside `MenuItem` storing expiry date (packaged), prep time (cooked), or an array of up to 3 component item IDs (combo), alongside `id` , `name` , `price` , `stock` , `sold_qty` , and `revenue` .
- **Critical Decisions:** (1) Stock updates validate `qty > 0 && qty <= item->stock` before deducting (for combos, all component stocks are verified atomically first). (2) Expiry warning converts dates to total days or checks year/month/day difference ≤ 2 days. (3) Inventory array and item count are encapsulated in a `CanteenStore` struct passed by pointer rather than globals.

```
PSEUDOCODE RecordSale(store, item_id, qty):
  item ← FindItemByID(store, item_id)
  IF item == NULL OR qty ≤ 0 OR item.stock < qty THEN
      PRINT "Sale rejected: insufficient stock or invalid item"
      RETURN FALSE
  END IF
  item.stock ← item.stock - qty
  item.sold_qty ← item.sold_qty + qty
  item.revenue ← item.revenue + (qty * item.price)
  RETURN TRUE

PSEUDOCODE WarnExpiring(store, today):
  FOR i = 0 TO store.count - 1 DO
      IF store.items[i].type == PACKAGED THEN
          days_left ← DaysBetween(today, store.items[i].info.expiry)
          IF days_left ≥ 0 AND days_left ≤ 2 THEN
              PRINT "EXPIRY ALERT: ", store.items[i].name, " expires in ", days_left, "
day(s)"
          END IF
      END IF
  END FOR
```

Stage 5 — Complete Working Implementation & Complexity Analysis

```
#include <stdio.h>
#include <string.h>

typedef struct { int d, m, y; } Date;
typedef struct { int hr, min; } Time;
typedef enum   { PACKAGED, COOKED, COMBO } ItemKind;

typedef struct {
    int id;
    char name[25];
    float price;
    int stock;
    int sold_qty;
    float revenue;
    ItemKind kind;
    union {
        Date expiry;
        Time prepared;
        int combo_ids[3];
    } detail;
} MenuItem;

typedef struct {
    MenuItem items[50];
    int count;
} Canteen;

int to_days(Date dt) { return dt.y * 365 + dt.m * 30 + dt.d; }

void warn_expiring(const Canteen *c, Date today) {
    for (int i = 0; i < c->count; i++) {
        if (c->items[i].kind == PACKAGED) {
            int diff = to_days(c->items[i].detail.expiry) - to_days(today);
            if (diff >= 0 && diff <= 2)
                printf("[EXPIRY WARNING] %s expires in %d day(s)!\n", c->items[i].name,
diff);
        }
    }
}

int record_sale(Canteen *c, int id, int qty) {
    for (int i = 0; i < c->count; i++) {
        if (c->items[i].id == id) {
            if (qty <= 0 || c->items[i].stock < qty) {
                printf("[REJECTED] %s: requested %d, available %d\n",
                       c->items[i].name, qty, c->items[i].stock);
                return 0;
            }
            c->items[i].stock -= qty;
            c->items[i].sold_qty += qty;
            c->items[i].revenue += qty * c->items[i].price;
            printf("[SOLD] %d x %s (Rs %.2f)\n", qty, c->items[i].name, qty * c-
>items[i].price);
            return 1;
        }
    }
    return 0;
}
```

Array vs. Linked List Complexity Analysis: In a contiguous array of structures, appending takes `O(1)` time, searching by ID takes `O(n)` time, and deleting an item requires `O(n)` element shifts, with zero pointer overhead and superior cache locality. In a singly linked list, inserting at the head takes `O(1)` time without shifting, while searching and deleting by ID still require `O(n)` pointer traversal; additionally, each node incurs 8 bytes of extra memory for the `next` pointer plus dynamic heap allocation overhead.